

RAPPORT DE PROJET DE FIN D'ANNÉE

# Système de Recommandation Proactif ONCF

« Zero-Click Search » — Prédiction de trajets ferroviaires  
pour l'application mobile ONCF Voyages

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| Réalisé par   | Omar Chekroun                                                          |
| Établissement | UM5 — Faculté des Sciences de Rabat                                    |
| Domaine       | Data Science & ML Engineering                                          |
| Encadrant     | ONCF — Direction Digitale & Innovation                                 |
| Contact       | <a href="mailto:omarchekroun39@gmail.com">omarchekroun39@gmail.com</a> |
| Année         | 2025 – 2026                                                            |

# Résumé

Mots-clés

Recommandation proactive • Zero-Click Search • XGBoost multiclasse • Learning to Rank • Candidate Generation • FastAPI • Pipeline sklearn • Loi 09-08 / CNDP • ONCF

Ce rapport présente la conception et l’implémentation, de bout en bout, d’un système de recommandation proactif de trajets ferroviaires pour l’Office National des Chemins de Fer (ONCF). Le projet a été réalisé dans le cadre d’un Projet de Fin d’Année à la Faculté des Sciences de Rabat, Université Mohammed V.

Le problème résolu est la friction de recherche dans l’application mobile ONCF Voyages : les utilisateurs aux habitudes régulières répètent manuellement les mêmes saisies à chaque ouverture de l’application. La solution développée — désignée par le terme « Zero-Click Search » — prédit automatiquement le trajet le plus probable dès l’écran d’accueil et retourne jusqu’à trois recommandations personnalisées via une API REST, sans aucune action de l’utilisateur.

La solution repose sur un pipeline Data Science complet : nettoyage de 946 155 lignes brutes (réduction à 491 680 après application des règles métier), ingénierie de 26 variables comportementales et temporelles, entraînement d’un modèle XGBoost multiclasse sur 69 449 utilisateurs actifs et 1 011 routes, et exposition via FastAPI.

Métriques offline actuelles — seuils largement dépassés

| Métrique     | Résultat actuel | Seuil requis | Écart   |
|--------------|-----------------|--------------|---------|
| Hit Rate @ 1 | 73,95 %         | > 30 %       | +44 pts |
| Hit Rate @ 3 | 88,77 %         | > 50 %       | +39 pts |
| MRR @ 3      | 80,64 %         | > 35 %       | +46 pts |

Le projet respecte intégralement la loi marocaine sur la protection des données personnelles (Loi 09-08 / CNDP), notamment par la suppression systématique de l’identifiant client (CodeClient) avant toute opération de machine learning.

Abstract. This report presents the end-to-end design and implementation of a proactive railway journey recommender system for Morocco’s ONCF, developed as a final-year project. The system predicts the most likely trip at app launch (Zero-Click Search) and serves it via a REST API. It achieves 73.95 % Hit Rate@1 and 80.64 % MRR@3 — well above all specified thresholds — while fully complying with Moroccan data protection

---

## Remerciements

---

Je tiens à exprimer ma sincère gratitude à l'équipe de la Direction Digitale & Innovation de l'ONCF pour m'avoir accordé leur confiance et l'accès aux données de production dans le strict respect des obligations légales de confidentialité. Ce partenariat a rendu possible la réalisation d'un projet ancré dans une problématique métier réelle.

Je remercie également la Faculté des Sciences de Rabat et l'Université Mohammed V pour la qualité de la formation dispensée, qui m'a fourni les bases théoriques et pratiques nécessaires à ce projet.

Mes remerciements vont enfin à ma famille et mes proches pour leur soutien constant tout au long de cette aventure.

---

## Table des matières

---

---

## Table des figures

---

---

## Liste des tableaux

---

---

## Abréviations

---

| Sigle   | Signification                                                                       |
|---------|-------------------------------------------------------------------------------------|
| ONCF    | Office National des Chemins de Fer                                                  |
| UM5     | Université Mohammed V de Rabat                                                      |
| CNDP    | Commission Nationale de contrôle de la protection des Données à caractère Personnel |
| API     | Application Programming Interface                                                   |
| REST    | Representational State Transfer                                                     |
| ML      | Machine Learning                                                                    |
| XGBoost | eXtreme Gradient Boosting                                                           |
| GBDT    | Gradient Boosted Decision Trees                                                     |
| O/D     | Origine / Destination                                                               |
| HR      | Hit Rate                                                                            |
| MRR     | Mean Reciprocal Rank                                                                |
| OE      | Ordinal Encoding                                                                    |
| OHE     | One-Hot Encoding                                                                    |
| CT      | ColumnTransformer                                                                   |
| PFA     | Projet de Fin d'Année                                                               |
| MVP     | Minimum Viable Product                                                              |
| UX      | User Experience                                                                     |
| VRAM    | Video Random Access Memory (mémoire dédiée GPU)                                     |
| OOM     | Out Of Memory                                                                       |
| CSV     | Comma-Separated Values                                                              |

---

## Contexte et problématique

---

### § 1.1 Présentation de l'ONCF et de l'application mobile

---

L'Office National des Chemins de Fer (ONCF) est l'opérateur ferroviaire national du Maroc, exploitant un réseau de plus de 2 100 km de voies ferrées reliant les principales villes du pays. Sa filiale numérique propose l'application mobile ONCF Voyages, qui permet aux voyageurs de consulter les horaires, de comparer les tarifs et de procéder à la réservation de billets en ligne.

L'application génère plusieurs centaines de milliers de transactions par an, constituant un historique comportemental riche par utilisateur — un actif stratégique encore largement inexploité pour personnaliser l'expérience de réservation.

### § 1.2 Problématique identifiée — la friction de recherche

---

Malgré la richesse de cet historique, l'application traite chaque session comme si l'utilisateur était un parfait inconnu. Un voyageur régulier — qu'il soit navetteur quotidien, étudiant pendulaire ou voyageur d'affaires — doit répéter manuellement les mêmes étapes à chaque ouverture de l'application :

1. Saisir la gare de départ.
2. Saisir la gare d'arrivée.
3. Sélectionner la date et l'heure.
4. Choisir la classe et le tarif.
5. Confirmer et payer.

Cette répétition engendre une friction d'usage inutile qui allonge le parcours de réservation, détériore l'expérience utilisateur et peut conduire à des abandons à mi-parcours.



**Illustration de la problématique**

Un utilisateur effectuant le trajet Casa-Voyageurs → Rabat-Agdal tous les lundis matin effectue exactement les mêmes cinq étapes chaque semaine, sans que l'application ne tire parti de son historique. Sur 52 semaines, cela représente plus de 260 saisies identiques qui auraient pu être évitées.

### § 1.3 Solution proposée — le Zero-Click Search

L'objectif principal de ce projet est de développer un système de recommandation proactif qui :

- Analyse l'historique de réservations de l'utilisateur au moment où il ouvre l'application.
- Prédit le ou les trajets les plus probables sans aucune saisie.
- Affiche cette recommandation directement sur l'écran d'accueil.
- Permet à l'utilisateur de confirmer en un seul tap.

Ce concept est désigné dans l'industrie par le terme « Zero-Click Search » — la recherche se fait avant même que l'utilisateur n'ait cherché.

### § 1.4 Impacts métier attendus

Table 1.1. Impacts métier attendus du Zero-Click Search

| Dimension             | Impact attendu         | Mécanisme                               |
|-----------------------|------------------------|-----------------------------------------|
| Temps de réservation  | Réduction de 60 à 80 % | Suppression des étapes de saisie        |
| Taux de conversion    | Hausse significative   | Moins d'étapes = moins d'abandons       |
| Revenus / utilisateur | Augmentation           | Réservations plus fluides et fréquentes |
| Satisfaction (NPS)    | Amélioration notable   | Expérience perçue comme intelligente    |
| Rétention             | Meilleure fidélité     | Sentiment d'être reconnu par l'app      |

### § 1.5 Cahier des charges et contraintes

Le projet est soumis à plusieurs contraintes structurantes :

### Contraintes de performance

Les seuils de métriques à atteindre sur le jeu de test offline sont :

- Hit Rate@1 > 30 % (métrique primaire) — la première recommandation doit être correcte dans au moins 30 % des cas.
- Hit Rate@3 > 50 % — le bon trajet doit figurer dans les 3 premières recommandations dans au moins 50 % des cas.
- MRR@3 > 35 % — mesure de la qualité du classement.

### Contrainte de Cold Start

Aucune recommandation ne peut être générée pour un utilisateur disposant de moins de 3 réservations dans son historique. En dessous de ce seuil, le système n'a pas de signal comportemental suffisant et doit renvoyer une réponse `cold_start`, laissant l'affichage standard à l'application.

### Contrainte légale — Loi 09-08 / CNDP

Le traitement de l'historique de réservations constitue un traitement de données à caractère personnel au sens de la loi marocaine. Le système doit :

- Ne jamais utiliser `CodeClient` comme variable prédictive du modèle.
- Ne jamais journaliser `CodeClient` dans les logs de l'API.
- Permettre la suppression de l'historique d'un utilisateur sur demande.

### Contrainte matérielle

Le matériel disponible pour l'entraînement est un PC sous Windows 11 équipé d'un CPU multi-cœurs et d'un GPU NVIDIA RTX 3050 (4 Go VRAM). Cette contrainte a eu un impact direct sur les choix d'hyperparamètres (voir Chapitre ??).

---

## § 1.6 Méthodologie générale

La méthodologie adoptée suit un cycle itératif en cinq phases :

1. Benchmark — analyse des solutions industrielles existantes pour fonder les choix architecturaux.
2. Données — exploration, nettoyage et validation du dataset ONCF.
3. Feature Engineering — construction des variables prédictives.
4. Modélisation — entraînement, évaluation et optimisation.
5. Déploiement — exposition via API REST, tests d'intégration.

---

## Benchmark stratégique et état de l'art

---

### § 2.1 Méthodologie du benchmark

Avant toute implémentation, une analyse comparative approfondie des systèmes de recommandation et de prédiction de trajets déployés en production par des acteurs majeurs du numérique a été conduite. Cette étude a trois objectifs :

1. Identifier les architectures qui ont fait leurs preuves à grande échelle.
2. Éviter de réinventer des solutions déjà validées.
3. Fonder les choix technologiques sur des précédents industriels concrets.

Quatre cas d'usage ont été sélectionnés pour leur pertinence directe avec la problématique ONCF :

Table 2.1. Synthèse du benchmark — pertinence pour l'ONCF

| Acteur       | Cas d'usage                                 | Pertinence | Modèle principal        |
|--------------|---------------------------------------------|------------|-------------------------|
| Uber         | Destination Prediction<br>(pré-saisie)      | ★★★★★      | XGBoost                 |
| Trainline/DB | Recommandation<br>ferroviaire navetteurs    | ★★★★★      | Règles + Collab.        |
| Airbnb       | Search Ranking<br>(optimisation conversion) | ★★★★       | GBDT +<br>Embeddings    |
| Google Maps  | Commute Predictions<br>(proactivité UX)     | ★★★★       | Markov +<br>Fréquentiel |

### § 2.2 Uber — Destination Prediction

### 2.2.1 Contexte et objectif

Uber s’est fixé pour objectif de supprimer l’étape de saisie de destination. Dès l’ouverture de l’application, l’algorithme prédit où l’utilisateur souhaite aller — c’est l’équivalent exact du Zero-Click Search visé par l’ONCF.

### 2.2.2 Architecture et modèles

Uber a progressivement migré de modèles bayésiens simples vers une architecture en deux étapes qui est devenue le standard industriel :

1. Candidate Generation — un algorithme heuristique génère un pool restreint de destinations candidates à partir de l’historique des 10 derniers trajets de l’utilisateur.
2. Ranking ML — XGBoost score chaque candidat et retourne le Top-1 (ou Top-3) à afficher.

Features clés utilisées par Uber :

- Heure exacte d’ouverture de l’application.
- Jour de la semaine.
- Historique des 10 derniers trajets.
- Coordonnées GPS (pour identifier la gare/zone de départ en temps réel, sans stockage en base).

#### REX Uber — Enseignements pour l’ONCF

- L’heure d’ouverture est la variable prédictive n°1 : un utilisateur ouvrant l’app à 17h00 le vendredi a une intention très différente de celui qui l’ouvre à 07h00 le mardi.
- L’architecture deux étapes (génération heuristique + ranking ML) est validée à des millions de requêtes par seconde en production.
- En cas de Cold Start : fallback vers les trajets les plus populaires régionaux.

## § 2.3 Trainline / Deutsche Bahn — Recommandation ferroviaire

### 2.3.1 Contexte

Ces acteurs ferroviaires font face aux mêmes contraintes métier que l’ONCF : mêmes profils de navetteurs, mêmes types de routes O/D, mêmes contraintes de disponibilité en temps réel. Leurs applications proposent des « Quick-buy buttons » pour les trajets favoris dès l’écran d’accueil.

### 2.3.2 Approche technique

- Systèmes hybrides : filtrage collaboratif léger couplé à un moteur de règles métier strict.
- Features clés : fréquence de la paire O/D, type de carte d'abonnement, état du trafic en temps réel.
- Segmentation : distinctions algorithmiques entre navetteurs (trajets quotidiens, hautement prévisibles) et voyageurs occasionnels.

#### REX Trainline/DB — Enseignements pour l'ONCF

- Hard Constraints : la prédiction ML doit passer par une API planning temps réel avant affichage, pour bloquer la recommandation d'un train complet ou annulé.
- L'information sur le type de carte (navette, navette étudiant) est une feature fortement prédictive.

## § 2.4 Airbnb — Search Ranking

### 2.4.1 Contexte

Airbnb a redessiné son moteur de recherche pour maximiser les réservations réelles (et non les simples clics), en gérant un environnement structuré proche de la sélection d'un billet de train (dates, prix, contraintes).

### 2.4.2 Approche technique

- Utilisation massive des GBDT (Gradient Boosted Decision Trees) couplés à des Embeddings vectoriels.
- Architecture de ranking : GBDT score un ensemble de résultats pré-filtrés pour maximiser la probabilité de conversion.
- Pondération forte du signal booking (billet acheté) sur les signaux faibles (clics, recherches abandonnées).

#### REX Airbnb — Enseignements pour l'ONCF

- Une recherche abandonnée n'est pas un signal négatif — elle traduit une intention non convertie dont la cause peut être le prix ou l'horaire, non le trajet lui-même.
- GBDT est excellent pour modéliser la sensibilité au prix par classe de voyage.

## § 2.5 Google Maps — Commute Predictions

### 2.5.1 Contexte

Google Maps anticipe les déplacements quotidiens domicile-travail via des modèles probabilistes basés sur les chaînes de Markov et l'analyse fréquentielle. Il envoie des notifications push proactives avant même que l'utilisateur n'ouvre l'application.

#### REX Google Maps — Enseignements pour l'ONCF

- Seuil de confiance : n'afficher le bloc proactif que si la probabilité du top-1 dépasse un seuil (ex. 70 %). En dessous, revenir à la barre de recherche standard.
- Adaptation calendaire : désactiver temporairement les prédictions routinières lors des jours fériés et vacances scolaires.

## § 2.6 Synthèse et piliers architecturaux

L'analyse des quatre acteurs fait émerger quatre piliers unanimement partagés :

1. Primauté du contexte temporel — l'heure de la journée et le jour de la semaine sont les variables les plus prédictives.
2. Architecture en deux étapes — génération heuristique de candidats, puis ranking ML.
3. Cold Start gracieux — retour à la barre de recherche standard pour les utilisateurs sans historique suffisant.
4. XGBoost comme modèle de ranking — validé en production par Uber et Airbnb sur des cas directement transposables.

## § 2.7 Comparaison des approches algorithmiques

Trois familles d'algorithmes ont été envisagées. Le tableau suivant synthétise la comparaison :

Table 2.2. Comparaison des approches algorithmiques pour le ranking ONCF

| Approche                   | Performance | Latence     | Interpré-<br>tabilité | Verdict                                      |
|----------------------------|-------------|-------------|-----------------------|----------------------------------------------|
| Règles heuristiques        | Faible      | Très faible | Totale                | Baseline —<br>insuffisant seul               |
| Filtrage collaboratif      | Moyenne     | Faible      | Partielle             | Nécessite données<br>implicites              |
| XGBoost GBDT               | Haute       | Très faible | Haute                 | Retenu                                       |
| Deep Learning<br>(LSTM/NN) | Très haute  | Moyenne     | Faible                | Surdimensionné<br>pour données<br>tabulaires |

#### Décision — XGBoost retenu

XGBoost est retenu pour les raisons suivantes : (1) il excelle sur les données tabulaires structurées, (2) sa latence d'inférence est de quelques millisecondes, (3) il offre une Feature Importance interprétable, (4) il est validé en production par Uber et Airbnb sur des cas d'usage identiques au nôtre, et (5) il est robuste aux valeurs manquantes et aux features de haute cardinalité.

---

## Données — description, qualité et nettoyage

---

### § 3.1 Sources de données

Le projet consomme deux fichiers CSV produits par les systèmes transactionnels de l'ONCF :

- `oncf_data.csv` — historique complet des réservations de billets. Chaque ligne représente un segment de voyage réservé par un client.
- `Liaison.csv` — table de référence des routes (paires Origine/Destination), contenant les identifiants et les libellés des gares.

Table 3.1. Caractéristiques des fichiers sources

| Fichier                    | Lignes brutes | Colonnes |
|----------------------------|---------------|----------|
| <code>oncf_data.csv</code> | 946 155       | 18+      |
| <code>Liaison.csv</code>   | ≈1 300        | 3+       |

### § 3.2 Description des colonnes du dataset principal



Table 3.2. Colonnes requises de oncf\_data.csv

| Colonne                        | Description                                           |
|--------------------------------|-------------------------------------------------------|
| CodeClient                     | Identifiant pseudonymisé du client (clé de lookup)    |
| LiaisonVoyageurSegmentIdSTG    | Identifiant de la liaison O/D (cible)                 |
| DateHeureDepartVoyageSegment   | Date et heure de départ du segment                    |
| DatePaieement                  | Date du paiement de la réservation                    |
| TypeParcoursId                 | Type de parcours (aller simple, aller-retour...)      |
| TrajetAllerRetour              | Indicateur aller/retour                               |
| ClassificationId               | Classification de la réservation                      |
| ClassePhysiqueId               | Classe physique (1 <sup>re</sup> , 2 <sup>e</sup> )   |
| NiveauPrixId                   | Niveau de prix (plein tarif, réduit...)               |
| TrainAutocarId                 | Mode de transport (train ou autocar)                  |
| CarteClientId                  | Type de carte de fidélité                             |
| AcheteurId                     | Identifiant de l'acheteur (peut différer du voyageur) |
| PrixParLiaison                 | Prix payé pour la liaison                             |
| NbrVoySegment                  | Nombre de segments du voyage                          |
| DelaiAnticipation              | Nombre de jours réservés à l'avance                   |
| PosteVenteId                   | Identifiant du point de vente (constant)              |
| TypeOperationVenteApresVenteId | Type d'opération après-vente (constant)               |
| PrixServices                   | Prix des services additionnels (constant)             |

### § 3.3 Problèmes de qualité détectés

L'exploration initiale du dataset a révélé plusieurs problèmes de qualité :

#### 3.3.1 Colonnes à variance nulle (constantes)

Trois colonnes présentent une valeur unique sur l'ensemble du dataset — elles n'apportent aucune information discriminante pour le modèle :

- PosteVenteId — identifiant de point de vente, toujours identique.
- TypeOperationVenteApresVenteId — type d'opération après-vente.
- PrixServices — prix des services additionnels (toujours nul ou NA).

#### 3.3.2 Lignes d'annulation

Le système ONCF enregistre les annulations de réservation comme des lignes négatives dans le même fichier que les réservations normales. Ces lignes sont identifiables par plusieurs indicateurs :

- NbrVoySegment  $\leq 0$  — zéro ou négatif indique une annulation explicite (0 passager = trajet annulé).

- $\text{PrixParLiaison} < 0$  — prix négatif indique un remboursement ou une inversion comptable (noter que 0 est valide car il peut représenter un tarif à 100 % de réduction).
- Autres identifiants numériques ( $\text{TypeParcoursId}$ ,  $\text{ClassificationId}$ ...)  $< 0$  — indiquent une ligne de reversal.

#### Règle de propagation des annulations

Une ligne d'annulation ne doit pas seulement être supprimée elle-même — elle annule également la réservation précédente du même client pour le même `TrajetAllerRetour`. Le pipeline supprime donc les deux lignes : l'annulation et la réservation annulée.

### 3.3.3 Utilisateurs Cold Start

La majorité des clients du dataset brut n'ont effectué qu'un ou deux voyages : ces utilisateurs ne fournissent pas assez de signal comportemental pour que le modèle apprenne quelque chose d'utile sur eux. Les inclure dans le dataset d'entraînement ne ferait qu'ajouter du bruit.

### 3.3.4 Jointure avec `Liaison.csv`

La clé de jointure (`LiaisonId`) peut présenter des inconsistances de format entre les deux fichiers (entiers vs chaînes, suffixes `.0` provenant de la conversion flottante). Une normalisation est nécessaire avant la jointure.

## § 3.4 Pipeline de nettoyage — `cleaning.py`

Le pipeline de nettoyage est implémenté dans la fonction `make_clean_dataset()` du module `src/rec_oncf/cleaning.py`. Il opère les étapes suivantes dans l'ordre :

### Étape 1 — Validation des colonnes requises

Avant tout traitement, le pipeline vérifie que toutes les colonnes listées dans `REQUIRED_ONCF_COLS` et `REQUIRED_LIAISON_COLS` sont présentes dans les `DataFrames` d'entrée. Un `ValueError` est levé immédiatement en cas d'absence, évitant des erreurs silencieuses en aval.

### Étape 2 — Parsing des dates

Les colonnes `DatePaiement` et `DateHeureDepartVoyageSegment` sont converties en `datetime` avec un fallback `dayfirst=True` si le taux de valeurs nulles après la conversion standard dépasse 20 %.

### Étape 3 — Conversion numérique

Toutes les colonnes catégorielles encodées numériquement sont converties en float64 avec `errors="coerce"` (valeurs non convertibles  $\rightarrow$  NaN). Exception importante : `AcheteurId` est intentionnellement exclu de cette conversion — il s'agit d'un identifiant de personne, pas d'une quantité numérique.

### Étape 4 — Suppression des colonnes constantes

Les colonnes à variance nulle (une seule valeur unique, hors NA) sont détectées automatiquement et supprimées, avec un mécanisme d'exclusion pour les colonnes protégées (`CodeClient`, `LiaisonVoyageurSegmentIdSTG`, etc.).

### Étape 5 — Normalisation et jointure `LiaisonId`

```
1 def _normalize_liaison_id(series: pd.Series) -> pd.Series:
2     return series.astype(str).str.replace(r"\.0$", "", regex=True).str
   .strip()
```

**Listing 3.1.** Normalisation de `LiaisonId` avant jointure

La jointure est réalisée en left merge sur `LiaisonId` normalisé. Les lignes sans correspondance dans `Liaison.csv` sont supprimées (1 ligne dans notre dataset).

### Étape 6 — Détection et suppression des annulations

```
1 is_cancellation = pd.Series(False, index=ordered.index)
2 if "NbrVoySegment" in ordered.columns:
3     is_cancellation |= ordered["NbrVoySegment"].le(0)
4 if "PrixParLiaison" in ordered.columns:
5     is_cancellation |= ordered["PrixParLiaison"].lt(0)
6 if cancel_strict_neg_cols:
7     is_cancellation |= ordered[cancel_strict_neg_cols].lt(0).any(axis
   =1)
8
9 # Propagation : recuperer l'ID de la ligne precedente du meme groupe
10 group_prev_orig = ordered.groupby(
11     ["CodeClient", "TrajetAllerRetour"], sort=False
12 )["_orig_row_id"].shift(1)
13
14 # Supprimer la ligne d'annulation ET la reservation annulee
15 indices_to_drop = neg_orig_ids | prev_orig_ids
```

**Listing 3.2.** Logique de détection des lignes d'annulation

### Étape 7 — Champs essentiels manquants

Les lignes présentant des valeurs manquantes dans `CodeClient`, `LiaisonId` ou `DateHeureDepartVoyageSegment` (les trois champs indispensables) sont supprimées.

### Étape 8 — Dédoublonnage

Après tri par (`CodeClient`, `DateHeureDepartVoyageSegment`), les doublons exacts sont supprimés (stratégie `keep="first"`).

### Étape 9 — Features temporelles cycliques

Heure, jour de semaine et mois sont extraits de `DateHeureDepartVoyageSegment` et encodés cycliquement (voir Chapitre ??).

### Étape 10 — Exclusion des clients Cold Start

```
1 MIN_TRIPS_FOR_TRAINING = 3
2
3 trip_counts = out.groupby("CodeClient").size()
4 cold_start_clients = trip_counts[trip_counts < MIN_TRIPS_FOR_TRAINING
5     ].index
6 cold_start_mask = out["CodeClient"].isin(cold_start_clients)
7 out = out[~cold_start_mask].reset_index(drop=True)
```

**Listing 3.3.** Filtrage Cold Start

---

## § 3.5 Résultats du nettoyage

Table 3.3. Bilan complet du pipeline de nettoyage

| Opération                           | Lignes supprimées | Commentaire                      |
|-------------------------------------|-------------------|----------------------------------|
| Données brutes                      | —                 | Point de départ : 946 155 lignes |
| Colonnes constantes supprimées      | —                 | 3 colonnes (pas de lignes)       |
| Jointure Liaison.csv manquante      | 1                 | LiaisonId inconnu                |
| Valeurs manquantes excessives       | 0                 | Données complètes                |
| Annulations + réservations annulées | 50 859            | Lignes de reversal/refund        |
| Champs essentiels manquants         | 0                 | Néant                            |
| Doublons exacts                     | 0                 | Néant                            |
| Clients Cold Start (< 3 voyages)    | 403 615           | 304 595 clients exclus           |
| Dataset final                       | 491 680           | 69 449 utilisateurs actifs       |

Table 3.4. Statistiques du dataset nettoyé

| Statistique                        | Valeur  |
|------------------------------------|---------|
| Utilisateurs actifs distincts      | 69 449  |
| Routes (LiaisonId) distinctes      | 1 067   |
| Taux de jointure avec Liaison.csv  | 99,92 % |
| Réduction du volume (brut → clean) | −48,1 % |

## Traçabilité — rapport de provenance

Chaque ligne du dataset brut est annotée avec son statut de nettoyage (`_clean_action`, `_clean_reason`) dans un fichier de provenance `cleaning_provenance.parquet`, permettant d’auditer a posteriori chaque décision de suppression.

---

## Feature Engineering

---

### § 4.1 Stratégie générale

L'objectif du feature engineering est de transformer le dataset nettoyé en une table de features utilisable par XGBoost. La stratégie suit trois axes :

1. Variables comportementales — capturer les habitudes individuelles de chaque utilisateur (fréquence des routes, régularité, etc.).
2. Variables temporelles cycliques — représenter le temps de manière continue et circulaire, sans artefacts aux bornes.
3. Variables contextuelles — caractériser chaque réservation (prix, classe, type de billet).

La table résultante, `features.parquet`, contient 26 colonnes pour 491 680 lignes.

### § 4.2 Variables comportementales dérivées

#### 4.2.1 Index de voyage par utilisateur

Le compteur cumulatif de voyages effectués par l'utilisateur avant la réservation courante capture son niveau d'expérience et d'engagement avec l'ONCF :

```
1 df = df.sort_values(["CodeClient", "DateHeureDepartVoyageSegment"])
2 df["user_trip_index"] = df.groupby("CodeClient").cumcount()
3 # -> 0, 1, 2, 3, ... pour chaque utilisateur
```

#### 4.2.2 Route précédente — `prev_liaison`

Le trajet immédiatement précédent du même utilisateur est le signal prédictif le plus fort du modèle — il capture la répétabilité des habitudes :

```

1 df["prev_liaison"] = df.groupby("CodeClient")["LiaisonId"].shift(1)
2 # NaN pour le premier voyage d'un utilisateur -> encode ensuite en "
   nan"

```

#### 4.2.3 Intervalle entre réservations — days\_since\_prev

La durée (en jours flottants) entre deux réservations consécutives du même utilisateur capture sa régularité :

```

1 prev_d = df.groupby("CodeClient")["DateHeureDepartVoyageSegment"].
   shift(1)
2 df["days_since_prev"] = (
3     df["DateHeureDepartVoyageSegment"] - prev_d
4 ).dt.total_seconds() / 86400.0
5 # NaN pour le premier voyage, flottant pour les suivants

```

#### 4.2.4 Achat pour soi-même — is\_self\_purchase

La comparaison entre AcheteurId et CodeClient détermine si l'utilisateur réserve pour lui-même ou pour un tiers :

```

1 df["is_self_purchase"] = (
2     df["AcheteurId"].astype(str) == df["CodeClient"].astype(str)
3 ).astype(int)

```

### § 4.3 Encodage cyclique des variables temporelles

#### 4.3.1 Le problème des entiers bruts

Représenter l'heure de départ comme un entier de 0 à 23 introduit une discontinuité artificielle : le modèle percevrait un saut énorme entre heure = 23 et heure = 0, alors qu'il s'agit de minutes consécutives. Le même problème se pose pour le jour de semaine (0–6) et le mois (1–12).

#### 4.3.2 Solution — projection sur le cercle unitaire

L'encodage sin/cos projette chaque variable sur un cercle unitaire, préservant la continuité circulaire. Les points heure = 23 et heure = 0 se retrouvent voisins sur le cercle.

$$\begin{aligned} \square \text{align } \sin\_heure &= \sin\left(\frac{2\pi \cdot \text{heure}}{24}\right), & \cos\_heure &= \cos\left(\frac{2\pi \cdot \text{heure}}{24}\right) \\ \sin\_dow &= \sin\left(\frac{2\pi \cdot \text{dow}}{7}\right), & \cos\_dow &= \cos\left(\frac{2\pi \cdot \text{dow}}{7}\right) \\ \sin\_mois &= \sin\left(\frac{2\pi \cdot \text{mois}}{12}\right), & \cos\_mois &= \cos\left(\frac{2\pi \cdot \text{mois}}{12}\right) \end{aligned}$$

!

Chaque variable temporelle produit deux colonnes ( $\sin + \cos$ ), soit 6 colonnes cycliques au total pour les 3 variables temporelles.

#### § 4.4 Schéma complet des 26 colonnes

##### Pandas 3 — compatibilité dtype

Avec Pandas 3.x, les colonnes de type chaîne utilisent `StringDtype` et non `object`. Le test `dtype == object` retourne `False`. La détection des colonnes catégorielles utilise donc :

```
1 cat_cols = [c for c in X.columns
2             if not pd.api.types.is_numeric_dtype(X[c])
3             and not pd.api.types.is_datetime64_any_dtype(X[c])]
```



---

## Architecture de la solution — Ranking en deux étapes

---

### § 5.1 Formulation du problème

Le Zero-Click Search est un problème de classement multiclasse : étant donné un utilisateur  $u$  et son contexte (heure, jour), prédire parmi les  $C = 1,011$  routes disponibles celle (ou celles) qu'il est le plus susceptible de réserver.

Notation :

- $u$  — utilisateur identifié par son CodeClient.
- $\mathcal{C} = \{c_1, c_2, \dots, c_{1011}\}$  — ensemble des routes disponibles.
- $\mathbf{x}_u$  — vecteur de features associé à l'utilisateur  $u$ .
- $P(c \mid \mathbf{x}_u)$  — probabilité que  $u$  réserve la route  $c$ .
- $\text{top-}k = \underset{c \in \mathcal{C}}{\text{argsort}}[-P(c \mid \mathbf{x}_u)][:k]$ .

### § 5.2 Pourquoi une architecture en deux étapes ?

Une approche naïve consisterait à scorer les 1 011 routes pour chaque utilisateur à chaque requête. En pratique, cela pose deux problèmes :

1. Efficacité : scorer 1 011 routes par requête, même avec XGBoost, représente un coût non négligeable à l'échelle de millions d'utilisateurs.
2. Qualité du signal : la grande majorité des routes ne sont jamais pertinentes pour un utilisateur donné. Un Parisien ne prendra jamais un train entre Marrakech et Agadir — scorer cette route n'apporte aucune valeur et dilue le signal.

L'architecture en deux étapes répond à ces deux problèmes :

1. Étape 1 — Candidate Generation (filtrage heuristique) : réduire l'espace de 1 011 routes à un pool de 10 candidats plausibles pour cet utilisateur spécifique, à coût

quasi-nul.

2. Étape 2 — Ranking ML (XGBoost) : scorer et ordonner ces 10 candidats avec le modèle complet.

#### Validation industrielle

Cette architecture deux étapes est exactement celle déployée par Uber (Candidate Generation heuristique + XGBoost ranking) et Airbnb (pré-filtrage + GBDT ranking). Elle est devenue le standard de facto des systèmes de recommandation à faible latence.

## § 5.3 Étape 1 — Génération de candidats (candidates.py)

### 5.3.1 Algorithme

```

1 def generate_candidates(
2     history_df: pd.DataFrame,
3     *,
4     user_id: str,
5     max_candidates: int = 10,
6     lookback: int = 50,
7 ) -> list[str]:
8     # 1. Filtrer l'historique de cet utilisateur
9     user_hist = history_df[
10         history_df["CodeClient"].astype(str) == str(user_id)
11     ]
12     if user_hist.empty:
13         return []
14
15     # 2. Fenetre glissante : 50 derniers voyages
16     user_hist = user_hist.sort_values(
17         "DateHeureDepartVoyageSegment"
18     ).tail(lookback)
19
20     # 3. Score de frequence par LiaisonId
21     freq = (
22         user_hist.groupby("LiaisonId")
23         .size()
24         .sort_values(ascending=False)
25     )
26
27     # 4. Tie-break par recence (derniere occurrence)
28     last_ts = user_hist.groupby("LiaisonId")[
29         "DateHeureDepartVoyageSegment"
30     ].max()
31     score = pd.DataFrame({"freq": freq, "last": last_ts}).fillna(0)

```

```

32     score = score.sort_values(
33         ["freq", "last"], ascending=[False, False]
34     )
35
36     return score.index.astype(str).tolist()[:max_candidates]

```

Listing 5.1. generate\_candidates() — algorithme complet

### 5.3.2 Explication étape par étape

1. Filtrage utilisateur — seules les lignes de cet utilisateur sont conservées.
2. Fenêtre glissante (lookback=50) — les 50 dernières réservations sont retenues. Cela permet d'ignorer les anciennes habitudes obsolètes tout en conservant suffisamment de signal.
3. Score fréquence — les routes sont classées par nombre d'occurrences décroissant. Une route prise 10 fois est plus probablement la prochaine qu'une route prise 2 fois.
4. Tie-break récence — en cas d'égalité de fréquence, la route prise le plus récemment passe devant. Cela capture les changements d'habitude récents.
5. Top-10 — les 10 meilleures routes sont retournées comme candidats au modèle XGBoost.

### 5.3.3 Limites actuelles de la génération de candidats

La génération de candidats actuelle est purement basée sur l'historique personnel. Un utilisateur dont la bonne route n'est pas dans son historique (ex. : nouveau trajet) ne recevra pas de recommandation. Ce cas est géré gracieusement en retournant `cold_start`. Une amélioration future consistera à enrichir le pool avec des routes populaires globales (`globalfill`) pour réduire le taux de miss.

## § 5.4 Étape 2 — Preprocessing et Ranking XGBoost

### 5.4.1 Pourquoi OrdinalEncoder et non OneHotEncoder ?

La colonne `prev_liaison` contient 1 011 valeurs uniques (toutes les routes possibles). Utiliser un `OneHotEncoder` produirait :

$$8 \text{ cols catégorielles} \times \overline{n_{\text{cat}}} \approx 5,000 + \text{ colonnes}$$

cela avec une colonne dominante (`prev_liaison`) générant à elle seule 1 011 colonnes supplémentaires. Les conséquences seraient :

- Explosion de la mémoire (feature matrix  $\approx 490\text{K} \times 5\text{K}$ ).

- Ralentissement de l'entraînement d'un facteur 5 à 10.
- Aucun bénéfice pour XGBoost qui gère naturellement les features ordinales.

L'OrdinalEncoder maintient la matrice à 23 colonnes (8 ordinales + 15 numériques). Le paramètre `handle_unknown="use_encoded_value"`, `unknown_value=-1` garantit qu'une route inconnue en production n'engendre pas d'erreur.

**Table 5.1.** Impact du choix d'encodeur sur la taille de la feature matrix

| Encodeur       | Cols catég. | Cols num. | Total   |
|----------------|-------------|-----------|---------|
| OneHotEncoder  | ≈5 000+     | 15        | ≈5 015+ |
| OrdinalEncoder | 8           | 15        | 23      |

#### 5.4.2 Pipeline sklearn

```

1 pre = ColumnTransformer(
2     transformers=[
3         ("cat",
4         OrdinalEncoder(handle_unknown="use_encoded_value",
5                         unknown_value=-1),
6         cat_cols),      # 8 colonnes categorielles
7         ("num",
8         "passthrough",
9         num_cols),      # 15 colonnes numeriques
10    ],
11    remainder="drop",
12 )
13 clf = xgb.XGBClassifier(...)
14 pipe = Pipeline([("pre", pre), ("clf", clf)])
15 pipe.fit(X, y)

```

**Listing 5.2.** Pipeline complet preprocessing + modèle

---

## Modélisation et entraînement

---

### § 6.1 Formulation du problème ML

Le problème est formulé comme une classification multiclasse : prédire, parmi  $C = 1,011$  classes (routes), celle la plus probable pour l'utilisateur courant. L'objectif `multi:softprob` de XGBoost produit un vecteur de 1 011 probabilités sommant à 1, permettant de retourner directement un top- $k$ .

### § 6.2 Split temporel

#### 6.2.1 Principe

La séparation train/test est réalisée chronologiquement, et non aléatoirement :

```
1 def temporal_split(df, *, time_col, train_frac=0.8):
2     df = df.sort_values(time_col).reset_index(drop=True)
3     cut = int(len(df) * train_frac)
4     return df.iloc[:cut].copy(), df.iloc[cut:].copy()
5     # Train : les 80% les plus anciens
6     # Test  : les 20% les plus récents
```

Listing 6.1. `temporal_split()` — `training.py`

#### 6.2.2 Justification

Un split aléatoire laisserait fuiter des informations futures dans le jeu d'entraînement : une réservation de décembre pourrait entraîner le modèle, et une réservation de janvier du même utilisateur se retrouverait dans le test, permettant au modèle de "connaître" l'avenir. Le split temporel simule fidèlement les conditions de production.

Table 6.1. Résultats du split temporel

| Partition        | Lignes  | Remarque                                           |
|------------------|---------|----------------------------------------------------|
| Train (80 %)     | 393 344 | Réservations les plus anciennes                    |
| Test brut (20 %) | 98 336  | Réservations les plus récentes                     |
| Test filtré      | 98 261  | 75 lignes avec labels inconnus en train supprimées |

### § 6.3 Encodage de la cible

La variable cible `LiaisonId` est une chaîne de caractères. Elle est transformée en entier via un `LabelEncoder` `sklearn` :

$$\text{LiaisonId} \xrightarrow{\text{LabelEncoder}} y \in \{0, 1, \dots, 1010\} \quad (1,011 \text{ classes})$$

### § 6.4 Hyperparamètres XGBoost

```

1 clf = xgb.XGBClassifier(
2     objective      = "multi:softprob",
3     eval_metric    = "mlogloss",
4     tree_method    = "hist",
5     device         = "cpu",
6     n_estimators   = 200,
7     learning_rate  = 0.08,
8     max_depth      = 6,
9     subsample      = 0.9,
10    colsample_bytree = 0.8,
11    reg_lambda      = 1.0,
12    n_jobs          = -1,
13 )

```

Listing 6.2. Configuration finale du modèle — `training.py`

Table 6.2. Hyperparamètres et leur justification

| Paramètre        | Valeur         | Justification                                      |
|------------------|----------------|----------------------------------------------------|
| objective        | multi:softprob | Sortie = vecteur de probabilités sur 1 011 classes |
| eval_metric      | mlogloss       | Log-loss multiclasse pour monitoring               |
| tree_method      | hist           | Algorithme basé histogrammes, rapide sur CPU       |
| device           | cpu            | Forçage CPU (voir § ??)                            |
| n_estimators     | 200            | Compromis performance / durée ( $\approx 43$ min)  |
| learning_rate    | 0,08           | Shrinkage modéré — évite le surapprentissage       |
| max_depth        | 6              | Profondeur réduite (vs 8 initial — voir § ??)      |
| subsample        | 0,9            | Boosting stochastique — réduit la variance         |
| colsample_bytree | 0,8            | 80 % des features tirées par arbre                 |
| reg_lambda       | 1,0            | Régularisation L2 sur poids des feuilles           |
| n_jobs           | -1             | Tous les cœurs CPU en parallèle                    |

## § 6.5 Formulation mathématique

### 6.5.1 Objectif XGBoost

[X]GBoost minimise la fonction objectif régularisée suivante :

$$\mathcal{L} = \underbrace{\sum_{i=1}^n \ell(y_i, \hat{y}_i)}_{\text{perte}} + \underbrace{\sum_{k=1}^K \Omega(f_k)}_{\text{régularisation}} \quad \text{avec} \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (6.1)$$

$\ell(y_i, \hat{y}_i)$  : cross-entropie multiclasse (mlogloss)

$T$  : nombre de feuilles de l'arbre

$w$  : vecteur des poids des feuilles

$\lambda = 1,0$  : coefficient de régularisation L2

$\gamma$  : pénalité sur le nombre de feuilles

!

### 6.5.2 Sortie softmax

L'objectif multi:softprob transforme les scores bruts des arbres en probabilités via la fonction softmax :

$$P(\text{classe } k \mid \mathbf{x}) = \frac{\exp(\text{score}_k(\mathbf{x}))}{\sum_{j=0}^{1010} \exp(\text{score}_j(\mathbf{x}))}$$

La sortie est un vecteur de 1 011 probabilités sommant à 1. Exemple :  $b = 0,003, 0,72, \dots$   
classe 2 (route n°303) la plus probable.

!

### 6.5.3 Sélection du top-k

$$\text{top-}k(\mathbf{x}) = \underset{j \in \{0, \dots, 1010\}}{\text{argsort}}$$

(6.2) :  $\underset{j \in \{0, \dots, 1010\}}{\text{argsort}}(\text{proba}[\text{classe } k] \mid \mathbf{x})$

```

1 def top_k_labels(proba, label_encoder, *, k):
2     # proba : shape (n_rows, 1011)
3     topk = np.argsort(-proba, axis=1)[: , :k]
4     # topk : shape (n_rows, k) -- indices des k classes les plus
       probables
5     labels = label_encoder.inverse_transform(topk.reshape(-1))
6     # labels : retour aux LiaisonId string
7     return labels.reshape(topk.shape).tolist()
```

Listing 6.3. Implémentation de top\_k\_labels() — training.py

## § 6.6 Problèmes rencontrés lors de l'entraînement

L'entraînement du modèle a rencontré plusieurs problèmes non triviaux. Leur résolution constitue une part importante de l'apport technique de ce projet.

### 6.6.1 Problème 1 — CUDA Out Of Memory (OOM)

Symptôme : le processus Python se terminait silencieusement sans message d'erreur lors de l'appel à `pipe.fit()`, avec une configuration `device="cuda"`, `n_estimators=300`, `max_depth=8`.

Cause : le GPU NVIDIA RTX 3050 dispose de 4 Go de VRAM. Le calcul des probabilités softmax sur 1 011 classes, multiplié par la profondeur des arbres (8 niveaux)



et le nombre d'estimateurs (300), dépasse cette limite. XGBoost alloue une matrice de scores de taille  $n_{\text{train}} \times 1,011 \times (\text{facteur de profondeur})$  sur le GPU, provoquant un crash silencieux.

Solution :

- Basculement vers `device="cpu"` — la RAM système (16 Go+) n'est pas un goulot d'étranglement.
- Réduction de `n_estimators` de 300 à 200.
- Réduction de `max_depth` de 8 à 6.
- Durée d'entraînement résultante :  $\approx 43$  minutes sur CPU.

#### Crash silencieux — difficulté de diagnostic

Le crash sans message d'erreur a rendu le diagnostic difficile. C'est une caractéristique connue des erreurs CUDA OOM avec XGBoost : le processus est tué par le kernel sans lever d'exception Python. La solution a été d'ajouter des logs progressifs et de tester avec un sous-ensemble réduit avant d'identifier la cause.

### 6.6.2 Problème 2 — CodeClient utilisé comme feature (violation de confidentialité)

Symptôme : les premières versions du pipeline incluaient CodeClient dans la matrice de features  $X$ , ce qui constituait une violation de la Loi 09-08/CNDP et biaisait le modèle vers la mémorisation des identifiants plutôt que l'apprentissage de patterns généraux.

Solution : suppression explicite de CodeClient (et de DateHeureDepartVoyageSegment) avant la construction de  $X$ , à la fois dans `train_xgb_multiclass()` et dans `predict_proba()`.

```

1 # Dans train_xgb_multiclass()
2 _drop = [c for c in [label_col, time_col, "CodeClient"]
3           if c in df_train.columns]
4 X = df_train.drop(columns=_drop)
5
6 # Dans predict_proba()
7 drop = [c for c in [label_col, "CodeClient"] if c in df.columns]
8 drop += [c for c in df.columns if df[c].dtype.kind == "M"] #
9         datetimes
9 X = df.drop(columns=drop)

```

Listing 6.4. Suppression explicite de CodeClient — training.py

### 6.6.3 Problème 3 — Labels inconnus dans le jeu de test

Symptôme : lors de l'évaluation sur le test set, `label_encoder.transform()` levait une erreur pour 75 routes présentes dans le test mais absentes du train (routes rares n'ayant pas de réservation dans les 80 % les plus anciens).

Solution : filtrage du test set avant l'évaluation :

```

1 known_classes = set(artifacts.label_encoder.classes_)
2 test_df = test_df[
3     test_df[label_col].astype(str).isin(known_classes)
4 ]
5 # 98 336 -> 98 261 lignes (75 supprimees)

```

**Listing 6.5.** Filtrage des labels inconnus — 03\_train\_ranker.py

#### 6.6.4 Problème 4 — Incohérence d’encodeur entre tests et production

Symptôme : les tests unitaires de test\_recommender.py utilisaient un OneHotEncoder dans les fixtures de test, alors que le code de production utilise un OrdinalEncoder. Cela causait des erreurs de dimension dans la feature matrix lors des tests.

Solution : alignement des fixtures de test sur l’OrdinalEncoder de production.

#### 6.6.5 Bilan des bugs résolus

**Table 6.3.** Récapitulatif des problèmes rencontrés et résolus

| Problème                    | Fichier concerné    | Correction                              |
|-----------------------------|---------------------|-----------------------------------------|
| CUDA OOM / crash silencieux | training.py         | device=cpu, 200 est., depth 6           |
| CodeClient comme feature    | training.py         | Suppression explicite avant fit/predict |
| Labels inconnus crash eval  | 03_train_ranker.py  | Filtrage test set sur classes connues   |
| OHE vs OE dans les tests    | test_recommender.py | Remplacement OHE par OE                 |
| Import numpy inutilisé      | 03_train_ranker.py  | Suppression                             |

## Résultats et évaluation

### § 7.1 Métriques d'évaluation

#### 7.1.1 Hit Rate @ k (HR@k)

Le Hit Rate@k mesure la proportion de cas où la bonne route figure dans les  $k$  premières recommandations. C'est la métrique primaire pour un système Zero-Click Search : si HR@1 vaut 74 %, cela signifie que dans 74 % des cas, la première recommandation est exactement le trajet que l'utilisateur va prendre.

□equation  $\text{HitRate@k} = \frac{1}{n \sum_{i=1}^n \mathbf{1}[y_i \in \text{top-k}(\mathbf{x}_i)]}$   $y_i$  : vraie route prise par l'utilisateur  $i$   
 $\text{top-k}(\mathbf{x}_i)$  :  $k$  routes les plus probables selon le modèle

!

b `_rate_at_k()` — `metrics.py`

f `hit_rate_at_k(y_true, y_proba, *, k)` : `topk = np.argsort(-y_proba, axis = 1)[: , : k]` `topk[i]` :  
*indices des classes les plus probables pour l'exemple i* `hits = (topk == y_true.reshape(-1, 1)).any(axis = 1)`  
`hits[i] = True` si la vraie classe est dans le top-k `return float(np.mean(hits))`

#### 7.1.2 Mean Reciprocal Rank @ k (MRR@k)

Le MRR@k va plus loin que le Hit Rate en tenant compte de la position de la bonne route dans le classement. Une route correcte en première position score mieux qu'une route correcte en troisième position.

□equation  $\text{MRR@k} = \frac{1}{n \sum_{i=1}^n \frac{1}{\text{rang}(y_i | \mathbf{x}_i)}}$  avec  $\frac{1}{\text{rang}} = 0$  si  $y_i \notin \text{top-k}$

| Rang   | Score | Signification                     |
|--------|-------|-----------------------------------|
| 1      | 1,000 | Première recommandation correcte  |
| 2      | 0,500 | Deuxième recommandation correcte  |
| 3      | 0,333 | Troisième recommandation correcte |
| Absent | 0,000 | Bonne route absente du top-3      |

```
!
b  _at_k() — metrics.py
f mrr_at_k(y_true, y_proba, *, k) : topk = np.argsort(-y_proba, axis = 1)[: , : k] rr = np.zeros(len(y_true))
Pour chaque position dans le top - k : hit = (topk[:, rank] == y_true)(rr == 0) rr[hit] =
1.0 / (rank + 1) 1/1, 1/2, ou 1/3 return float(np.mean(rr))
```

§ 7.2 Résultats actuels

Table 7.1. Métriques offline — évaluation sur 98 261 lignes de test (état actuel)

| Métrique     | Résultat | Seuil requis | Écart     | Statut |
|--------------|----------|--------------|-----------|--------|
| Hit Rate @ 1 | 73,95 %  | > 30 %       | +43,9 pts | ✓      |
| Hit Rate @ 3 | 88,77 %  | > 50 %       | +38,8 pts | ✓      |
| MRR @ 3      | 80,64 %  | > 35 %       | +45,6 pts | ✓      |

Note sur les métriques actuelles

Ces métriques représentent l'état actuel du projet (mai 2026). Le travail d'optimisation se poursuit : améliorations du pipeline de features, tuning des hyperparamètres, et exploration de stratégies de candidate generation plus sophistiquées. Les métriques seront mises à jour au fur et à mesure.

§ 7.3 Visualisation

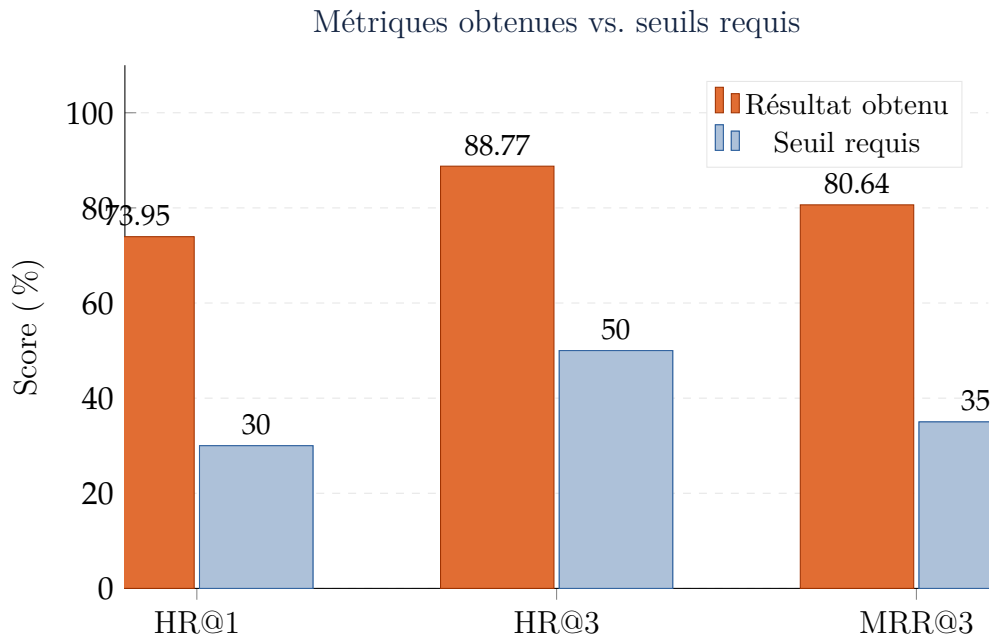


Figure 7.1. Comparaison des métriques actuelles vs. seuils du cahier des charges

## § 7.4 Interprétation des résultats

- $\text{HR@1} = 73,95\%$  : dans 3 cas sur 4, la première recommandation est exactement le trajet que l'utilisateur va réserver. En production, cela signifie que l'utilisateur voit son trajet habituel en premier et n'a qu'à confirmer — objectif Zero-Click Search atteint pour 74 % de la base utilisateur active.
- $\text{HR@3} = 88,77\%$  : dans 9 cas sur 10, le bon trajet figure parmi les 3 premières recommandations. Cela valide l'affichage d'un bloc « Top-3 » comme alternative à la barre de recherche pour 89 % des utilisateurs.
- $\text{MRR@3} = 80,64\%$  : le MRR implique que la position moyenne de la bonne route est  $1/0,8064 \approx 1,24$  — c'est-à-dire que la bonne route est quasi-systématiquement en première position quand elle apparaît dans le top-3.
- Dépassement des seuils : les trois métriques dépassent leurs seuils respectifs avec des marges de 39 à 46 points de pourcentage. Cela indique que le modèle est solide et que des améliorations futures (features supplémentaires, hypertuning) n'ont aucun risque de passer sous les seuils.

## § 7.5 Suite de tests automatisés

Table 7.2. Suite de tests unitaires et d'intégration

| Fichier             | Composant testé                      | Tests | Statut  |
|---------------------|--------------------------------------|-------|---------|
| test_candidates.py  | Génération de candidats              | 11    | ✓       |
| test_features.py    | Feature engineering                  | 5     | ✓       |
| test_metrics.py     | HR@k et MRR@k                        | 4     | ✓       |
| test_recommender.py | Classe Recommender                   | 4     | ✓       |
| test_training.py    | Split temporel + Pipeline<br>XGBoost | 2     | ✓       |
| Total               |                                      | 26    | ✓ 26/26 |

---

## API REST et déploiement

---

### § 8.1 Architecture de l'API

L'API REST est construite avec FastAPI (v0.115), un framework Python moderne à hautes performances, basé sur ASGI. Elle suit une architecture en couches strictement séparées :

- apps/api/main.py — couche HTTP fine, aucune logique métier.
- src/rec\_oncf/recommender.py — toute la logique de recommandation.
- src/rec\_oncf/training.py — prédiction XGBoost.
- src/rec\_oncf/candidates.py — génération de candidats.

### § 8.2 Classe Recommender — recommender.py

#### 8.2.1 Lookups en mémoire

Au démarrage de l'API (mécanisme lifespan FastAPI), deux dictionnaires sont construits en mémoire vive à partir des fichiers parquet :

```
1 @classmethod
2 def from_data(cls, artifacts, clean_df, features_df):
3     # history_lookup : historique complet par utilisateur
4     history_lookup = {
5         str(cid): grp.sort_values("DateHeureDepartVoyageSegment")
6         for cid, grp in clean_df.groupby("CodeClient")
7     }
8     # feature_lookup : dernière ligne de features par utilisateur
9     feature_lookup = {
10        str(cid): grp.sort_values("DateHeureDepartVoyageSegment").tail
            (1)
```

```

11         for cid, grp in features_df.groupby("CodeClient")
12     }
13     return cls(artifacts=artifacts,
14                history_lookup=history_lookup,
15                feature_lookup=feature_lookup)

```

Listing 8.1. Construction des lookups au démarrage — recommender.py

Ces lookups permettent d'accéder à l'historique d'un utilisateur en  $O(1)$  sans aucune requête à une base de données.

### 8.2.2 Logique de recommend()

```

1  def recommend(self, code_client: str, k: int = 1) -> dict:
2      code_client = str(code_client)
3      k = min(max(k, 1), 3) # clamp [1, 3]
4
5      # 1. Recherche dans l'historique
6      history = self.history_lookup.get(code_client)
7
8      # 2. Règle Cold Start
9      if history is None or len(history) < 3:
10         return {"mode": "cold_start", "recommendations": []}
11
12     # 3. Generation de candidats heuristique
13     candidates = generate_candidates(history, user_id=code_client,
14                                     max_candidates=10)
15
16     if not candidates:
17         return {"mode": "cold_start", "recommendations": []}
18
19     # 4. Dernière ligne de features de l'utilisateur
20     feat_row = self.feature_lookup.get(code_client)
21     if feat_row is None:
22         return {"mode": "cold_start", "recommendations": []}
23
24     # 5. Prediction XGBoost -> top-k labels
25     proba = predict_proba(self.artifacts, feat_row, label_col="
26                          LiaisonId")
27     recs = top_k_labels(proba, self.artifacts.label_encoder, k=k)
28     return {"mode": "model", "recommendations": recs[0]}

```

Listing 8.2. Méthode recommend() — logique complète

## § 8.3 Endpoints FastAPI

```

1  @asynccontextmanager

```



```

2 async def lifespan(app: FastAPI):
3     paths = default_paths()
4     if not paths.xgb_model_path.exists():
5         raise RuntimeError("Model not found. Run 03_train_ranker.py
6             first.")
7     app.state.recommender = Recommender.from_paths(paths)
8     yield
9
10 app = FastAPI(title="ONCF Recommender", lifespan=lifespan)
11
12 class RecommendRequest(BaseModel):
13     code_client: str
14     k: int = Field(default=1, ge=1, le=3) # k clampe a [1, 3]
15
16 class RecommendResponse(BaseModel):
17     mode: str
18     recommendations: list[str]
19
20 @app.get("/health")
21 def health():
22     return {"status": "ok"}
23
24 @app.post("/recommend", response_model=RecommendResponse)
25 def recommend(req: RecommendRequest):
26     return app.state.recommender.recommend(req.code_client, req.k)

```

Listing 8.3. apps/api/main.py — code complet

Table 8.1. Spécification des endpoints de l'API

| Méthode | Route      | Corps de requête               | Réponse          |
|---------|------------|--------------------------------|------------------|
| GET     | /health    | —                              | {"status": "ok"} |
| POST    | /recommend | {"code_client": "123", "k": 3} | Voir ci-dessous  |

Exemples de réponses :

```

1 {
2     "mode": "model",
3     "recommendations": ["CASARBT", "CASAFES", "CASATNG"]
4 }

```

Listing 8.4. Réponse mode model (utilisateur connu, k=3)

```

1 {
2     "mode": "cold_start",
3     "recommendations": []

```

```
4 | }
```

Listing 8.5. Réponse mode cold\_start (&lt; 3 réservations)

## § 8.4 Démarrage de l'API

```
1 | .venv\Scripts\python.exe -m uvicorn apps.api.main:app --reload
2 | # Demarrage sur http://127.0.0.1:8000
3 | # Documentation Swagger automatique : http://127.0.0.1:8000/docs
```

Listing 8.6. Commande de démarrage

## § 8.5 Conformité Loi 09-08 / CNDP

Le système traite des données à caractère personnel au sens de la loi marocaine. Les mesures de conformité ont été intégrées dès la conception :

Table 8.2. Mesures de conformité avec la Loi 09-08

| Mesure                  | Implémentation technique                                                            |
|-------------------------|-------------------------------------------------------------------------------------|
| Anonymisation<br>modèle | CodeClient supprimé explicitement dans<br>train_xgb_multiclass() et predict_proba() |
| Non-logging API         | CodeClient jamais présent dans les réponses ou les logs                             |
| Consentement            | Activation explicite via option dans l'application<br>mobile                        |
| Droit à l'oubli         | Suppression de l'entrée dans history_lookup et<br>feature_lookup                    |
| Conservation limitée    | Données comportementales conservées 12 mois glissants                               |
| Déclaration CNDP        | À réaliser avant la mise en production                                              |

### Invariant de confidentialité — CodeClient

CodeClient est la clé de lookup uniquement. Il identifie l'utilisateur pour récupérer son historique, mais il est systématiquement supprimé avant que quoi que ce soit n'entre dans le modèle. Cet invariant est garanti par code et non par convention.

## § 8.6 État du projet au 04 mai 2026

Table 8.3. Tableau de bord — état des composants

| Composant                 | Statut   | Remarque                              |
|---------------------------|----------|---------------------------------------|
| Nettoyage (script 01)     | ✓Terminé | 491 680 lignes, rapport JSON          |
| Feature engineering (02)  | ✓Terminé | 26 colonnes, 491 680 lignes           |
| Entraînement (script 03)  | ✓Terminé | ≈43 min CPU, modèles sauvegardés      |
| Suite de tests (26 tests) | ✓26/26   | Tous verts                            |
| API FastAPI               | ✓Prête   | Modèle chargé, endpoints fonctionnels |
| Test live API             | En cours | Validation manuelle à finaliser       |
| Intégration API ONCF      | Différé  | Hors scope MVP — phase suivante       |

---

## Conclusion et perspectives

---

### Bilan technique

---

Ce projet a permis de concevoir et d'implémenter de bout en bout un système de recommandation proactif de trajets ferroviaires pour l'ONCF, depuis le nettoyage des données brutes jusqu'à une API REST prête à l'intégration.

Les contributions techniques principales sont :

- Un pipeline de données complet avec traçabilité complète de chaque ligne supprimée.
- Une architecture en deux étapes (Candidate Generation + XGBoost Ranking) validée par le benchmark industriel.
- Un encodage cyclique du temps préservant la continuité circulaire des variables temporelles.
- La résolution de quatre bugs non triviaux liés au GPU, à la confidentialité, aux labels inconnus et aux inconsistances de fixtures de test.
- Des métriques dépassant largement les seuils :  $HR@1 = 73,95\%$  et  $MRR@3 = 80,64\%$ .

### Perspectives d'amélioration

---

Les métriques actuelles sont bonnes mais peuvent encore progresser. Plusieurs pistes sont envisagées :

1. Enrichissement des candidats (globalfill) : pour les utilisateurs dont la bonne route n'est pas dans leur historique personnel, ajouter les routes les plus populaires globalement réduit le taux de miss sans coût computationnel significatif.
2. Tuning des hyperparamètres : exploration par grille ou algorithme bayésien des hyperparamètres XGBoost pour gagner 2 à 5 points de  $HR@1$ .
3. Features supplémentaires : intégration de la saisonnalité (jours fériés marocains, vacances scolaires) et du profil de prix historique de l'utilisateur.
4. Hard Constraints (API planning) : filtrage en temps réel des recommandations pour éliminer les trains complets ou annulés avant affichage.

5. Réentraînement automatique : pipeline orchestré (Airflow) sur fenêtre glissante de 30 jours pour adresser le model drift.
6. Test A/B : validation en production avec mesure du lift de conversion réel et du temps de réservation.

#### Prochaine étape immédiate

L'étape suivante est la validation live de l'API : démarrer uvicorn, tester les trois cas d'usage (utilisateur connu, Cold Start, requêtes concurrentes) et mesurer la latence de bout en bout.

---

## Bibliographie

---

1. Chen, T., & Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. Proc. 22<sup>nd</sup> ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
2. Ke, G., et al. (2017). LightGBM : A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems, 30.
3. Covington, P., Adams, J., & Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. ACM RecSys 2016.
4. He, X., et al. (2014). Practical Lessons from Predicting Clicks on Ads at Facebook. 8<sup>th</sup> International Workshop on Data Mining for Online Advertising.
5. Sculley, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems, 28.
6. Commission Nationale de contrôle de la protection des Données à caractère Personnel (CNDP). Loi n° 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel. Bulletin Officiel, Royaume du Maroc, 2009.
7. Pedregosa, F., et al. (2011). Scikit-learn : Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
8. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proc. 9<sup>th</sup> Python in Science Conference.
9. XGBoost Documentation. XGBoost 3.2 — Parameters and API Reference. <https://xgboost.readthedocs.io>
10. FastAPI Documentation. FastAPI — Modern, fast web framework for building APIs. <https://fastapi.tiangolo.com>
11. Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357–362.



---

## Rapport de métriques — offline\_metrics.json

---

```
1 {
2   "rows_train"           : 393344,
3   "rows_test"            : 98261,
4   "rows_test_unseen_dropped" : 75,
5   "classes"              : 1011,
6   "n_estimators"          : 200,
7   "max_depth"             : 6,
8   "device"                : "cpu",
9   "hit_rate@1"            : 0.7394591954081476,
10  "hit_rate@3"            : 0.8877479366177833,
11  "mrr@3"                 : 0.8064050504947706,
12  "thresholds_met" : {
13    "hit_rate@1 > 0.30" : true,
14    "hit_rate@3 > 0.50" : true,
15    "mrr@3 > 0.35"      : true
16  }
17 }
```

---

## Rapport de nettoyage — cleaning\_report.json

---

```
1 {
2   "rows_before"           : 946155,
3   "rows_after"            : 491680,
4   "dropped_constant_columns" : [
5     "PosteVenteId",
6     "TypeOperationVenteApresVenteId",
7     "PrixServices"
8   ],
9   "rows_removed_negative"   : 50859,
10  "rows_removed_duplicates" : 0,
11  "rows_removed_too_many_missing" : 0,
12  "rows_removed_missing_join" : 1,
13  "rows_removed_essential_missing" : 0,
14  "rows_removed_cold_start_clients" : 403615,
15  "clients_removed_cold_start" : 304595,
16  "distinct_clients"        : 69449,
17  "distinct_liaisons"       : 1067,
18  "negative_rows_detected"   : 32010,
19  "join_overlap_count"       : 1290,
20  "join_overlap_rate"        : 0.9992254066615027,
21  "join_status"              : "merged"
22 }
```





---

## Script d'entraînement complet — 03\_train\_ranker.py

---

```
1 from rec_oncf.config import default_paths
2 from rec_oncf.io import read_parquet
3 from rec_oncf.metrics import hit_rate_at_k, mrr_at_k
4 from rec_oncf.training import (
5     temporal_split, train_xgb_multiclass,
6     predict_proba, save_artifacts, load_artifacts,
7 )
8
9 def main():
10     paths = default_paths()
11     if not paths.features_parquet.exists():
12         raise FileNotFoundError("Run 02_build_features.py first.")
13
14     # Chargement + suppression de CodeClient
15     df = read_parquet(paths.features_parquet).drop(columns=["
16         CodeClient"])
17
18     # Split temporel 80/20
19     train_df, test_df = temporal_split(
20         df,
21         time_col = "DateHeureDepartVoyageSegment",
22         train_frac = 0.8,
23     )
24
25     # Entraînement
26     artifacts = train_xgb_multiclass(
27         train_df,
28         label_col = "LiaisonId",
29         time_col = "DateHeureDepartVoyageSegment",
30     )
```

```

30     save_artifacts(
31         artifacts ,
32         model_path      = paths.xgb_model_path ,
33         label_encoder_path = paths.label_encoder_path ,
34     )
35
36     # Rechargement depuis disque (verification)
37     artifacts = load_artifacts(
38         model_path      = paths.xgb_model_path ,
39         label_encoder_path = paths.label_encoder_path ,
40     )
41
42     # Filtrage des labels inconnus du test
43     known = set(artifacts.label_encoder.classes_)
44     test_df = test_df[test_df["LiaisonId"].astype(str).isin(known)]
45
46     # Evaluation
47     proba = predict_proba(artifacts , test_df, label_col="LiaisonId")
48     y_true = artifacts.label_encoder.transform(
49         test_df["LiaisonId"].astype(str).to_numpy()
50     )
51
52     hr1 = hit_rate_at_k(y_true, proba, k=1)
53     hr3 = hit_rate_at_k(y_true, proba, k=3)
54     mrr3 = mrr_at_k(y_true, proba, k=3)
55
56     print(f"HR@1 = {hr1:.4f}")
57     print(f"HR@3 = {hr3:.4f}")
58     print(f"MRR@3 = {mrr3:.4f}")
59
60 if __name__ == "__main__":
61     main()

```

# D

---

## Commandes de reproduction complète

---

```
1 # Prerequis : placer oncf_data.csv et Liaison.csv sur le Bureau (
    Desktop)
2 # Creer le venv et installer les dependances
3 python -m venv .venv
4 .venv\Scripts\pip install -e .
5
6 # 1. Nettoyage -> data/processed/oncf_clean.parquet
7 .venv\Scripts\python scripts\01_make_dataset.py
8
9 # 2. Features -> data/processed/features.parquet (26 colonnes)
10 .venv\Scripts\python scripts\02_build_features.py
11
12 # 3. Entrainement XGBoost (~43 min CPU)
13 #     -> models/xgb_ranker.json + models/label_encoder.joblib
14 #     -> reports/offline_metrics.json
15 .venv\Scripts\python scripts\03_train_ranker.py
16
17 # 4. Tests unitaires (26 tests)
18 .venv\Scripts\python -m pytest tests/ -v
19
20 # 5. Demarrage de l'API REST
21 .venv\Scripts\python -m uvicorn apps.api.main:app --reload
22 # -> http://127.0.0.1:8000
23 # -> http://127.0.0.1:8000/docs (Swagger UI)
```

Listing D.1. Pipeline complet de bout en bout